Geona Michelle NNM24IS085

OS_Report_Geona_Michelle_.docx

Plag Report

5B and UID

Nitte University

Document Details

Submission ID

trn:oid:::1:3688300690

Submission Date

Oct 23, 2026, 2:06 PM GMT+5:30

Download Date

Oct 23, 2026, 3:15 PM GMT+5:30

File Name

OS_Report_Geona_Michelle_.docx

File Size

240.9 KB

18 Pages

5,458 Words

29,215 Characters

7% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography

Match Groups

23 Not Cited or Quoted 5%
Matches with neither in-text citation nor quotation marks

10 Missing Quotations 2%
Matches that are still very similar to source material

0 Missing Citation 0%
Matches that have quotation marks, but no in-text citation

0 Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

6% Internet sources
1% Publications
3% Submitted works (Student Papers)

Match Groups

- **23** Not Cited or Quoted 5%
  Matches with neither in-text citation nor quotation marks
- **10** Missing Quotations 2%
  Matches that are still very similar to source material
- **0** Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

6% Internet sources
1% Publications
3% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Internet	
github.com		2%
2	Student papers	
University of Nebraska at Omaha		1%
3	Internet	
engineersofai.com		<1%
4	Internet	
vi.wikipedia.org		<1%
5	Internet	
gitelephant.cypresslab.net		<1%
6	Internet	
www.coursehero.com		<1%
7	Student papers	
Nitte University		<1%
8	Student papers	
University of Alabama at Birmingham		<1%
9	Internet	
code84.com		<1%
10	Publication	
Leblanc. "<![CDATA[Debugging Parallel Programs with Instant Replay]]>", IEEE Tr...		<1%

11 Internet

www.bartleby.com <1%

12 Student papers

Universidad Tecnológica de Bolívar,UTB <1%

13 Internet

www-dweb-cors.dev.archive.org <1%

14 Internet

par.nsf.gov <1%

15 Publication

Mohammad Shamsuddoha. "Integrated Supply Chain Model for Sustainable Man... <1%

16 Internet

archive.org <1%

NMAM INSTITUTE OF TECHNOLOGY, NITTE

(An Autonomous Institution affiliated to VTU, Belagavi)

Department of Information Science and Engineering

OPERATING SYSTEMS

Programming Assignment Report

on

AI-Assisted Synchronization Algorithms with Program-Generated HTML Simulation

Readers-Writers Problem and Dining Philosophers Problem

Submitted by

Geona Michelle

USN: NNM24IS085

GitHub: https://github.com/GeonaMichelle/OS_TASK2

Submitted to

Dr Jason Elroy Martis

Associate Professor

Department of Information Science and Engineering

NMAM Institute of Technology, Nitte

Table of Contents

1. Statement of Problem and Objectives of the Study....3
2. Readers-Writers: Using Semaphores....4
3. Readers-Writers: Using Monitor....5
4. Dining Philosophers: Using Semaphore....6
5. Dining Philosophers: Using Monitor....7
6. Logging of Event and Generating HTML....8
7. Observation, Verification Checks and Traces....9
8. Race Condition, Mutual Exclusion, Deadlocks, and Starvation....11
9. Difference Between Semaphore and Monitor, Limitation and Conclusion....13
10. HTML Screenshots from Simulation Generated....14
11. AI Use Summary....16
12. References....16
13. Statement of Academic Integrity....17

Abstract

For my submission on this task I have written four small Python programs for solving Readers-Writers and Dining Philosophers problems. For each problem I have used two approaches; first is the use of semaphores while the second approach makes use of a monitor implemented by a combination of locks and condition variables. During execution, the program will log all the important events, check the logs according to some set of safety properties, and finally produce an HTML page showing a replay of the current execution. In this document I will explain each solution, give the core code snippet and then give an example of one execution of each program with discussion about Race Conditions, Mutual Exclusion, Deadlocks and Starvation.

1. Statement of Problem and Objectives of the Study

However, without control, if several threads work with the same data, then the result can be absolutely wrong, but in different ways according to the operation of the scheduling algorithm. In order to study all these processes in more detail, I solved two classical problems in each of two ways: by means of **semaphores** [6] and by means of **monitors** [8]. Therefore, I have written four programs:

- The **Reader-Writers Problem** using **Semaphores**
- The **Reader-Writers Problem** using **Monitors**
- The **Dining Philosophers Problem** (4 philosophers) using **Semaphores**
- The **Dining Philosophers Problem** (4 philosophers) using **Monitors**

Objective of this research is to study mutual exclusion and coordination, deadlock prevention, and starvations possibilities. All programs are written in simple Python using standard `threading` package [1]. Additionally, all programs have a special logger mechanism that allows them to generate an HTML replay from the results of the program execution.

1.1 Project files

File	Purpose
`ReadersWritersSemaphore.py`	Two semaphores Readers-Writers (`mutex`, `wrt`), reader preference
`ReadersWritersMonitor.py`	Monitor Readers-Writers, writer preference
`DiningPhilosophersSemaphore.py`	Four philosophers, one semaphore per fork and one semaphore for the room of 3
`DiningPhilosophersMonitor.py`	For four philosophers, state array and condition for each philosopher
`readers_writers_semaphore.html`, `readers_writers_monitor.html`, `dining_philosophers_semaphore.html`, `dining_philosophers_monitor.html`	Run the programs again and show the pages produced by the programs
`README.md`	How to run the programs and what each program does

1.2 Settings used in the recorded runs

Program	Threads	Work per thread
Readers-Writers (semaphore)	4 readers, 2 writers	2 rounds each; a write adds 10
Readers-Writers (monitor)	5 readers, 2 writers	3 rounds each; a write adds 1
Dining Philosophers (semaphore)	4 philosophers, 4 forks	3 meals each; room semaphore = 3
Dining Philosophers (monitor)	4 philosophers, 4 forks	3 meals each

1.3 How to read this report

Sections 2 through 5 cover each of the programs separately, providing descriptions of their source code. Section 6 gives the information on recording of events and their conversion into HTML. Actual trace and all programs' verification are covered in Section 7. Sections 8 and 9 deal with the theory

and the comparison of semaphores and monitors, respectively. The screenshot, description of AI help, references and declaration are given at the bottom of the paper.

Running of the project. Run the project in VS Code by opening the folder and running the program, e.g. `python ReadersWritersSemaphore.py` within the folder (use `py` or `python3` if required). HTML will be automatically created right next to the program file and opened in a web browser; to disable this option, use `AUTO_OPEN_HTML = False`. The variables `NUM_READERS`, `NUM_WRITERS`, `ROUNDS`, `NUM_PHILOSOPHERS` and `MEALS` can be modified at the

2. Readers-Writers: Using Semaphores

Shared Resource: Integer `shared_value`. The readers only read it, and writers increment the `shared_value` by 10. The access to this variable is the critical section; many readers can be in this critical section, but not more than one writer.

Semaphores: `mutex` = 1 protects the counter `read_count`, and `wrt` = 1 guarantees the mutual exclusion to the shared resource. The first reader takes `wrt` in behalf of all readers, and the last one leaves it. Writers take `wrt` for their own needs to prevent the readers and writers.

This is a **reader preference** algorithm described in [7] by Courtois et al. The issue with this algorithm is that if writers come one after another, `wrt` is locked, and thus the waiting writer may wait forever [2]. For my simulation, I have 4 readers and 2 writers per iteration of two rounds.

Core code:

```
mutex = threading.Semaphore(1)        # protects read_count
wrt = threading.Semaphore(1)          # exclusive access to the shared resource

def reader(reader id):
    global read count
    for   in range(ROUNDS):
        mutex.acquire()                          # entry section
        try:
            read count += 1
            if read count == 1:                  # first reader blocks writers
                wrt.acquire()
        finally:
            mutex.release()

        value = shared value                     # critical section: readers share it
        time.sleep(random.uniform(0.4, 0.9))

        mutex.acquire()                          # exit section
        try:
            read count -= 1
            if read count == 0:                  # last reader lets writers in
                wrt.release()
        finally:
            mutex.release()

def writer(writer id):
    global shared value
    for   in range(ROUNDS):
        wrt.acquire()                            # exclusive access
        try:
            shared value += 10                   # each write adds 10
            time.sleep(random.uniform(0.4, 0.9))
        finally:
            wrt.release()
```

The final output for the experiment turned out to be 40, which means there were four updates of 10 each. In the comparison between the value of 40 and the last write which was successfully updated, any scenario of lost update could easily be detected.

2.1 Step-by-step working

- The reader initially acquires the `mutex` lock and then increments the value of `read_count`.
- If it's the first reader (`read_count`=1), then it also acquires `wrt`. Thus, from now on, no writer will be able to acquire `wrt`. Other readers will not do this and just pass through.
- The reader releases `mutex` and reads the value. Since it does not acquire `mutex` while reading the value, thus, some other readers may read the value simultaneously.
- On departure, the reader again acquires `mutex` lock and reduces the value of `read_count`. The last reader to depart (`read_count` = 0) releases the `wrt` lock.
- A writer simply acquires the `wrt` lock, modifies the value, and then releases the `wrt` lock. However, if some readers are still there, it has to wait for `wrt` lock.

2.2 Why it is correct, and where it is weak

How it is safe: The writer can enter in it only when the system has `wrt`, and the readers keep the system collectively as a group, and thus the possibility of overlap between reader-writer and writer-writer does not exist. **Shortcomings:** In case there is at least one reader always present, then `wrt` cannot be available to the writer, and thus the writer has to wait infinitely. It is a typical example of **starvation** problem where readers have the priority over the writers [4][7]. That is why I designed the other program.

3. Readers-Writers: Using Monitor

There is no monitor keyword in Python programming language, that's why `ReadersWriterMonitor` class was used in order to implement this functionality. There is one lock, two condition variables (`can_read` and `can_write`) and the state variables (`active_readers`, `writer_active` and `waiting_writers`). All methods operate under the protection of the lock, which means that only one thread can change the state [8].

In this implementation, **writers have the priority**. If there is an active writer or even writer who waits, then the new readers will wait. Writer has to wait until there is active reader or another writer. When some thread cannot continue the execution of its task, then it calls `wait()` method which unlocks the lock and suspends the thread. Thread wakes up and checks its condition in the while loop because the state can be changed by other threads. After finishing the writer, it wakes up the next waiting writer or all the readers if there is no waiting writer in this case. Number of writers and readers were set to 2 and 5 correspondingly, and number of rounds was 3. The number of writes is equal to 1.

Core code:

```
class ReadersWriterMonitor:
    def __init__(self):
        self.lock = threading.Lock()
        self.can_read = threading.Condition(self.lock)    # readers wait here
        self.can_write = threading.Condition(self.lock)   # writers wait here
        self.active_readers = 0
```


turnitin Page 9 of 22 - Integrity Submission Submission ID trn:oid:::1:3688300690

```
        self.writer active = False
        self.waiting writers = 0

    def start read(self, name):
        with self.lock:
            while self.writer active or self.waiting writers > 0:
                self.can read.wait()                    # writer preference
            self.active readers += 1

    def end read(self, name):
        with self.lock:
            self.active readers -= 1
            if self.active readers == 0:
                self.can write.notify()                 # last reader wakes one writer

    def start write(self, name):
        with self.lock:
            self.waiting writers += 1
            while self.writer active or self.active readers > 0:
                self.can write.wait()
            self.waiting writers -= 1
            self.writer active = True

    def end write(self, name):
        with self.lock:
            self.writer active = False
            if self.waiting writers > 0:
                self.can write.notify()                 # next writer goes first
            else:
                self.can read.notify all()              # no writer waiting: wake all readers

# in the writer thread
monitor.start write(name)
shared value += 1                                       # only one writer can be here
monitor.end_write(name)
```

Since the waiting writer will prevent the incoming reader from accessing him/her, a flow of readers will not manage to exclude a writer in future. The price to be paid for such an action is that the readers will have to wait for a long time.

3.1 Step-by-step working

- **Reader enters:** if there are any active writers or waiting writers, it executes `can_read.wait()` and sleeps. Otherwise, it increments the number of active readers and begins reading.
- **Reader exits:** decrements the number of active readers. If it is the last reader, it wakes up any waiting writers by executing `can_write.notify()`.
- **Writer enters:** increments the number of `waiting_writers` first, which prevents any other new readers from entering, and waits for no active readers or writers.
- **Writer exits:** sets `writer_active` to false. If there is any waiting writer, it goes ahead; otherwise, it wakes up all sleeping readers using `notify_all()`.

3.2 Why use a while loop rather than an if

Once the thread is awakened, it cannot get the monitor immediately. It must try for the lock once more, and at that point, there could be some other thread which has changed the state of the thread. The while loop helps the thread check its condition every time it is awakened; Therefore, it will not go forward based on this old-fashioned belief. It would be a tough one if it were employed.

3.3 Fairness

However, it is a fact that the policy according to which the readers should wait when the writers are also waiting is fair to the writers, but the same policy may prevent the readers from accessing the

system for a longer period of time if writers continue arriving. In the simulation performed, it was noted that the reader R3 was waiting till 0.484 s because the writer W2 was waiting.

4. Dining Philosophers: Using Semaphore

There are four philosophers sitting at a table in which there are four forks such that there is a fork in between every two philosophers. The philosopher i needs the fork i to his left and fork (i+1) % 4 to his right. Therefore, each fork would be used by two adjacent philosophers. The initial value of each fork would be 1. `Room`=3 can accommodate only three philosophers at a time.

In case of **deadlock**, all four philosophers possess one fork and wait for the other one (circular wait condition). However, since there are only three philosophers who can seize the forks, the circular wait condition cannot happen; hence at least one philosopher will be able to seize both forks, eat and release them. Since the semaphores in Python don't follow FCFS strategy, **starvation** is possible. That's why the program makes sure that all philosophers eat their meals. In my present execution, there are 4 philosophers eating 3 meals each..

Core code:

```
fork = [threading.Semaphore(1) for   in range(NUM PHILOSOPHERS)]   # one per fork
room = threading.Semaphore(NUM PHILOSOPHERS - 1)                   # N-1 places at the table

def philosopher(i):
    left = i
    right = (i + 1) % NUM PHILOSOPHERS
    for meal in range(1, MEALS + 1):
        time.sleep(random.uniform(0.1, 0.6))          # thinking

        room.acquire()                                # only N-1 may enter
        fork[left].acquire()                          # pick up left fork
        time.sleep(random.uniform(0.05, 0.2))         # makes the overlap visible
        fork[right].acquire()                         # pick up right fork

        time.sleep(random.uniform(0.3, 0.6))          # eating: holds both forks

        fork[left].release()                          # put down forks
        fork[right].release()
        room.release()                                # leave the room
```

This records who owns the fork at the time of picking it up and when it is put down. In this way, there will be consistency in who owns what fork on the printed page.

4.1 Step-by-step Operation

One day the philosopher thinks at an arbitrary point in time and subsequently gets hungry.

- The philosopher asks for a permit from the room. Only three are permitted; the fourth one is left outside as there will always be a free seat.
- While inside the room, the philosopher picks up his left fork, pauses for a bit, and picks up his right fork. The pause was intentional as this would give time to other philosophers to pick up their forks and make it evident during the replay.
- The philosopher eats by using both forks, puts them down, and exits the room.

4.2 The appropriateness of three philosophers

There being four forks and four philosophers, it is necessary for all of them to pick up left forks and then wait for the right forks so as to form a deadlock condition. However, in a case where there are

not more than three philosophers in the room, there would not be more than three left forks which they would pick up. There would be one left fork available for the philosopher near him to pick up, eat, and then release the forks.

5. Dining Philosophers: Using Monitor

The `DiningMonitor` class stores `state[i]` (THINKING, HUNGRY or EATING) for each philosopher, as well as one condition variable for each philosopher, using just one lock. A philosopher is able to call two methods: `pickup(i)` and `putdown(i)`.

The important condition can be seen in `_test(i)`: in case a hungry philosopher discovers that neither of his neighbors is eating, he is made EATING, meaning that both forks are picked up at the same time. Thus, the fact is that a philosopher never holds a fork but not another one, so there is **no hold-and-wait condition** for deadlock. A philosopher who puts down his forks will check his neighbors and wake them up if they are hungry.

Starvation may happen theoretically in case two neighbors take their turns forever; this is why the check if everyone has completed all his/her meals is done too.

Core code:

```
class DiningMonitor:
    def   init  (self):
        self.lock = threading.Lock()
        self.state = [THINKING] * NUM PHILOSOPHERS
        self.cond = [threading.Condition(self.lock) for   in range(NUM PHILOSOPHERS)]

    def  left(self, i):  return (i - 1) % NUM PHILOSOPHERS
    def  right(self, i): return (i + 1) % NUM PHILOSOPHERS

    def  test(self, i):
        # eat only if BOTH neighbours are not eating
        if (self.state[i] == HUNGRY
                and self.state[self. left(i)] != EATING
                and self.state[self. right(i)] != EATING):
            self.state[i] = EATING                # takes both forks in one step
            self.cond[i].notify()

    def pickup(self, i, meal):
        with self.lock:
            self.state[i] = HUNGRY
            self. test(i)
            while self.state[i] != EATING:        # wait() releases the lock while sleeping
                self.cond[i].wait()

    def putdown(self, i):
        with self.lock:
            self.state[i] = THINKING
            self. test(self. left(i))             # left neighbour may eat now
            self._test(self._right(i))            # right neighbour may eat now
```

5.1 Step-by-step working implementation

- `pickup(i)`**:** the philosopher makes himself HUNGRY and calls the function `_test(i)`. If none of his two neighbors is eating, he instantly assumes the state of EATING; otherwise, he waits for his own condition variable.
- **Eating:** the philosopher picks up both forks. As far as our model is concerned, the ownership of the fork is established when the owner begins to eat.

- `putdown(i)`: the philosopher becomes THINKING and checks out his two neighbors. The hungry neighbor that is able to eat is assigned the state EATING and is awakened.
- Every philosopher has his own condition variable, that is why only those neighbors that can be benefited are awakened, but not all.

5.2 Compared with the Semaphore version

Although the semaphore solution requires the use of an extra semaphore for the protection of the space, the monitor solution utilizes the safe property of `_test`. Another advantage that the monitor brings with it is that it facilitates more concurrency in terms of eating. Specifically, P0 and P2 were eating concurrently in the test run done, which is the highest level of concurrency possible given that there are four forks being used.

6. Logging of Event and Generating HTML

- Four programs implement the following procedure: run threads, call `record(...)` for every state change, create HTML, and view HTML on the page.
- A thread calls `record(...)` each time at some significant point during its execution. Function `record(...)` obtains the lock on `log_lock`, increments some counters, applies the rules, and adds event dictionaries to the events list. Event dictionary consists of the time passed from the start of the simulation in milliseconds, thread name, its state, and resource's actual state (number of readers, number of fork owners, or monitor variables).
- In the monitor program `record(...)` is called with monitor lock obtained; therefore, state stored in the event is correct.
- `make_html(...)` converts events into JSON using `json.dumps(...)`, and replaces the place marked by `EVENTS` marker in HTML template with this JSON data. HTML template includes JavaScript code that simulates execution.
- On the client side, the script reconstructs state after the first N events: thread cards with colors, resource state, timeline with playhead, event log, and buttons Play, Step, Reset, and speed selection.

The identical recorded data powers the automatic **PASS/FAIL** checks which can be found on both the terminal and page sides.

Since the page is built based on the events that have been recorded, it will depict the actual experience and not the animation of the run. This explains why every run will generate a unique page..

6.1 What one recorded event contains

Field	Meaning
`time`	Time lapse in milliseconds since the beginning of the program execution
`who`	Thread name such as R3, W1 or P2
`state`	Each State moves from one to another like waiting, reading, writing, hunger or eating
`detail`	Like "blocked because the writer is active."
`resource state`	Number of readers, number of writers inside, value they hold, number of forks, number of rooms in monitor variables

6.2 Reason for the lock on the log

Several threads call the `record()` method at the same time. Without using a lock, the entries could be corrupted and the counters incremented incorrectly. The validation routine won't work in this case. For this reason, another lock `log_lock` is used to make sure that all the entries are safe. Logically speaking, it was necessary to ensure that this lock was independent of other algorithm locks.

6.3 The replay page

Thread Cards, Resource, Timeline and Event Log are available on the page. Play starts the events at the selected speed, Step advances one step forward and Reset resets to the beginning. As the entire event list is stored on the page in data format, it operates offline, runs in any web browser and is also able to be submitted as a regular file.

7. Observation, Verification Checks and Traces

The traces have been obtained from actual recorded traces (where time is given in seconds). There is a different sequence every time; therefore, your trace will be different.

7.1 Readers-Writers Using Semaphore

Four people read collectively, and the writer had to wait until they all left before he joined them.

```
0.151s W2 -> waiting
0.153s W2 -> writing (value 0->10)
0.253s R3 -> waiting
0.784s W2 -> finished writing (value 10)
0.818s W2 -> idle (releases the resource)
0.837s R3 -> reading (value=10)
0.837s R1 -> reading (value=10)
0.838s R2 -> reading (value=10)
0.838s R4 -> reading (value=10)
1.155s W2 -> waiting (readers hold the resource)
1.605s R4 -> idle (last reader releases)
1.605s W1 -> writing
```

7.2 Readers-Writers Using Monitor

R3 was delayed solely on account of the writer waiting, thus fulfilling the **writer preference** criterion.

```
0.058s W1 -> waiting
0.058s W1 -> writing (value 0->1)
0.110s R1 -> waiting (held back: writer active)
0.173s R4 -> waiting (held back: writer active)
0.414s W1 -> finished writing
0.414s R2 -> reading (value=1)
0.414s R4 -> reading (value=1)
0.415s R1 -> reading (value=1)
0.416s W2 -> waiting
0.484s R3 -> waiting (held back: writer waiting)
0.881s W2 -> writing (value 1->2)
```

7.3 Dining Philosophers Using Semaphore

```
0.269s P3 -> hungry
0.270s P3 -> acquiring (entered room, 1 of 3)
0.270s Fork F3 taken by P3
0.289s P0 -> acquiring (room 2 of 3)
0.291s Fork F0 taken by P0
0.420s Fork F1 taken by P0
0.420s P0 -> eating
0.464s P1 -> acquiring (room 3 of 3, room full)
0.567s P2 -> hungry (must wait outside the room)
0.962s P0 -> releasing F0 and F1, leaves the room
0.963s P3 takes F0 and eats, P2 enters the room
```

7.4 Dining Philosophers Using Monitor

```
0.181s P2 -> hungry
0.183s P2 -> eating (takes F2 and F3 together)
0.211s P3 -> hungry (P2 eating, must wait)
0.291s P0 -> eating (takes F0 and F1; P0 and P2 eat together)
0.316s P1 -> hungry (P0 and P2 eating, must wait)
0.730s P2 -> thinking (puts down F2 and F3)
0.882s P0 -> thinking (puts down F0 and F1)
0.883s P3 -> eating (woken by P0)
0.883s P1 -> eating (woken by P0)
```

Summary of the four recorded runs:

Program	Events	Result
Readers-Writers (semaphore)	48	PASSED, total marks 40
Readers-Writers (monitor)	84	Eight of eight check marks achieved, total score six out of five simultaneous readings possible
Dining Philosophers (semaphore)	96	7 of 7 tick marks cleared, maximum 3 people in room, all meals consumed
Dining Philosophers (monitor)	36	All 7 checks have passed, max 2 meal times eating together, all meals eaten

7.5 Verification checks

After the threading process is completed, the program compares the events recorded to the rules and prints out either **PASS** or **FAIL**. This will be a FAIL if there is any violation of any of the rules.

Program	Rules checked
RW semaphore	Value at the end = Writes done (4 writes of 10 = 40) Final answer = PASS
RW monitor	Only one writer at a time; Never reader and writer at the same time; Never negative counters; All readers and writers finish all the loops; Final value is equal to expected and the last writing; Monitor exits properly (8 tests)
DP semaphore	Two forks never held; never do neighbours eat together; eats only if both forks are available; maximum 3 philosophers in room; all meals eaten; no deadlock occurs; forks free after meal eating is completed (7 test cases)

turnitin Page 15 of 22 - Integrity Submission Submission ID trn:oid:::1:3688300690

Program	Rules checked
DP monitor	Never two forks in one hand; never neighbours dine together; states of thinkers in proper order think, hungry, and eat; maximum two thinkers eating simultaneously; all dinners eaten; no deadlock occurs; all philosophers thinking in the end (seven tests)

7.6 Observations from the traces

- In case of the semaphore Readers-Writers the number of readers reading simultaneously reached 4 for 0.837 seconds, while the time that passed between arrival of the last reader and arrival of the writer W1 reached 1.605 seconds. It is an illustration of the reader preference behaviour.
- In case of the monitor Readers-Writers reader R3 stayed despite there being only one writer waiting but not writing; hence, it is the writer preference.
- In case of the semaphore Dining Philosophers the room was occupied by 3 philosophers of 3, and P2 stayed outside until P0 left the room. Thus, the deadlock guard is shown.
- In case of the monitor Dining Philosophers two philosophers without neighbours dined simultaneously, while the hungry philosopher woke up as soon as his neighbour left both forks.
- The sequence of events differs each time because of random scheduler and sleep, but the rules were always followed in all observed cases.

8. Race Condition, Mutual Exclusion, Deadlocks, and Starvation

Race condition. Race condition occurs when there are simultaneous modifications of shared data between threads. For instance, when `read_count += 1` is executed without `mutex`, or both writers read the same value without writing it back, one of them will definitely overwrite another one. In these programs, `mutex`, `wrt`, and monitor lock prevent race condition.

Mutual Exclusion. Mutual exclusion means that only one writer can access the shared data at a time, and no reader and writer can be inside at the same time. This rule is ensured by `wrt` in the semaphore program, and by monitor lock and its conditions in the monitor program.

Philosophers. Every fork has a single owner at any one time and its neighbours never dine together, and the automatic checks confirm this.

Deadlock. To have a deadlock, mutual exclusion, hold and wait, no preemption, and circular wait must be present together [9]. Circular wait is violated in the semaphore version of the philosophers by setting `room` = 3, while hold and wait is violated in the monitor version by picking up both forks at once.

Starvation. Since the semaphore Readers-Writers algorithm favors readers, the writers might starve if more readers continue coming [4]. In case of monitor, writers have priority over readers, thus readers can wait for a long time. For Dining Philosophers, neither solution guarantees FIFO, which means that starvation is theoretically possible.

turnitin Page 16 of 22 - Integrity Submission Submission ID trn:oid:::1:3688300690

8.1 The four deadlock conditions applied to Dining Philosophers

Condition	Semaphore version	Monitor version
Mutual exclusion	Present: fork has one owner	Nowadays: the fork is owned by an individual
Hold and wait	Now: holding left fork while waiting for right	Broken: both forks are picked up simultaneously
No preemption	Nowadays: a fork will be dropped by its user only	Now
Circular wait	BROKEN: Capacity for room number 3 prevents all 4 from waiting	Present: cannot be formed because nobody is waiting with the fork

8.2 Starvation summary

Program	Who may starve	Reason
RW semaphore	Writers	`wrt` remains locked as readers continue to overlap
RW monitor	Readers	Potential readers hold their breath when the author does.
DP semaphore	Any philosopher in theory	Python makes no first-in-first-out guarantee
DP monitor	A philosopher with neighbours that alternate	Neighbors may continue taking the forks.

9. Difference Between Semaphore and Monitor, Limitation and Conclusion

Aspect	Semaphore	Monitor
Structure	functions to acquire/release scattered throughout thread's code	Encapsulate the lock, condition variable and data inside a class
Waiting	Thread is waiting for semaphore's status	Thread keeps waiting on the condition variable in a while loop.
Resource control	Programmer pairs each acquire with a release	Access control by monitoring techniques
Risk of error	Order is critical; otherwise, there will be no synchronization.	Encapsulation means that verifying the rules is going to be easier.
Abstraction level	low-level and flexible	More abstract thinking and structure
Readers-Writers here	Tendency of the reader r	Writer's choice
Dining Philosophers here	Semaphores room (N-1) avoid deadlock	Array State and `_test()` work on both forks

Conclusion. Semaphores are extremely flexible; nevertheless, each acquire and release call has to be implemented by the programmer, and one wrong call will render everything useless. The monitor holds the data, the lock, and the conditions in one unit, and according to my experience, this made it significantly easier to reason about the program [8][10].

turnitin Page 17 of 22 - Integrity Submission Submission ID trn:oid:::1:3688300690

9.1 Limitations

- Passing these constraints during the execution run means that the constraints are met in the run. It does not mean that the programs are correct under all possible schedules.
- The Python threads are subject to one interpreter lock. Thus, the synchronization is done correctly; however, there is no real parallelism.
- Sleeps added to the program just to show overlapping are artificial. They are useful for illustrating the principle but change the time from real-time.
- Python semaphores and conditions are not FIFO. Thus, the fairness of synchronization cannot be achieved.

9.2 Possible future work

- Include the unbiased Readers Writers algorithm in which the order of arrival will decide who goes ahead next so that neither readers nor writers face starvation.
- Re-run the programs several times and describe how many times each thread has waited the maximum amount.
- Increase the number of philosophers and find out how the capacity of the room affects N-1.
- Create a button in the replay feature to save the log of the events in a CSV file.

9.3 Overall conclusion

Each of the four programs has accomplished their intended purpose; there were no conflicts between readers and writers, there was never the conflict of two forks owned by two different individuals, there was never the scenario where two neighbors were having dinner together, and none of the iterations ended up causing a deadlock. The experience of writing all these programs has made me realize that the tricky part is not the amount of coding but who can wait and for how long.

turnitin Page 18 of 22 - Integrity Submission Submission ID trn:oid:::1:3688300690

10. HTML Screenshots from Simulation Generated

Below are the images representing the output of the HTML simulations produced by the four different programs.

10.1 Readers-Writers Using Semaphore

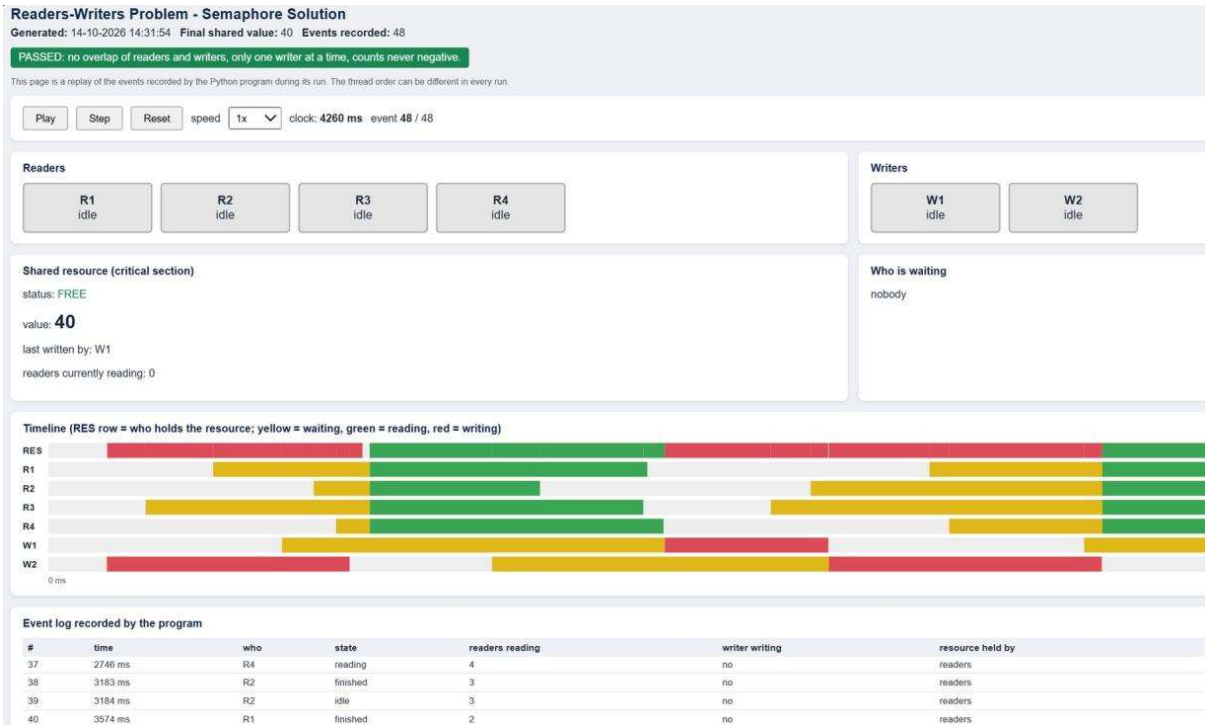


Figure 1: *readers_writers_semaphore.html (event 48 of 48, final value 40)*

10.2 Readers-Writers Using Monitor

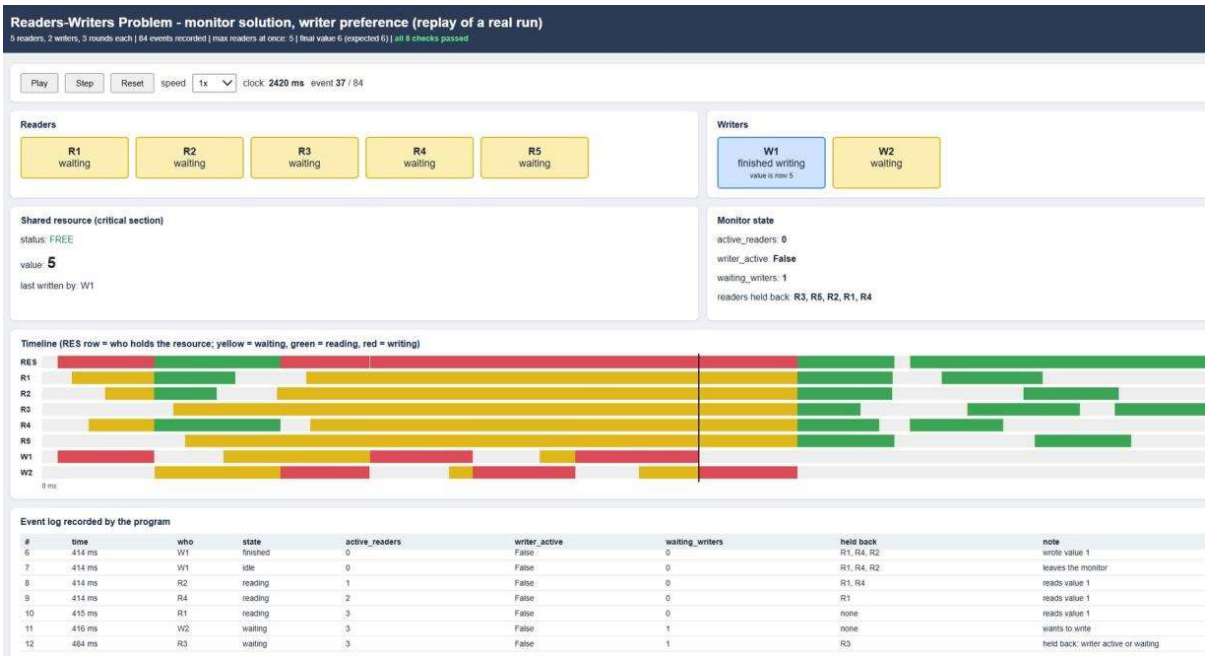


Figure 2: *readers_writers_monitor.html (event 37 of 84, writer preference in action)*

10.3 Dining Philosophers Using Semaphore

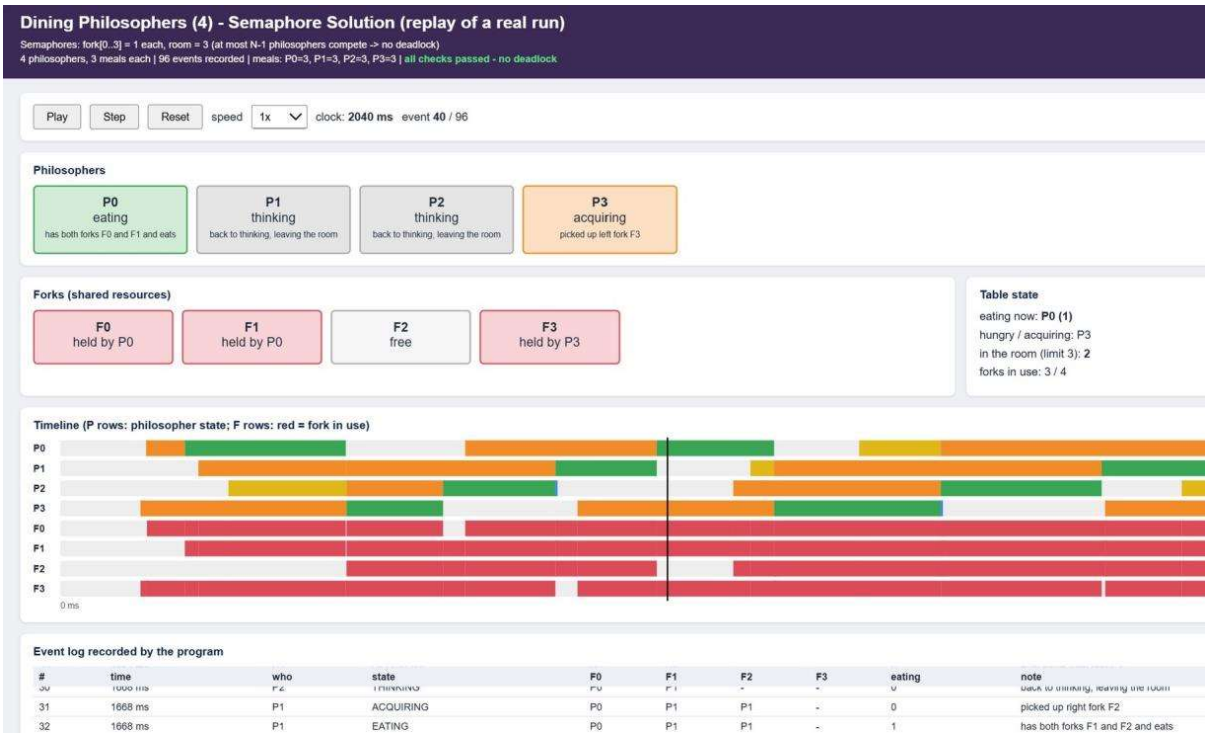


Figure 3: *dining_philosophers_semaphore.html (event 40 of 96, room limit 3)*

10.4 Dining Philosophers Using Monitor

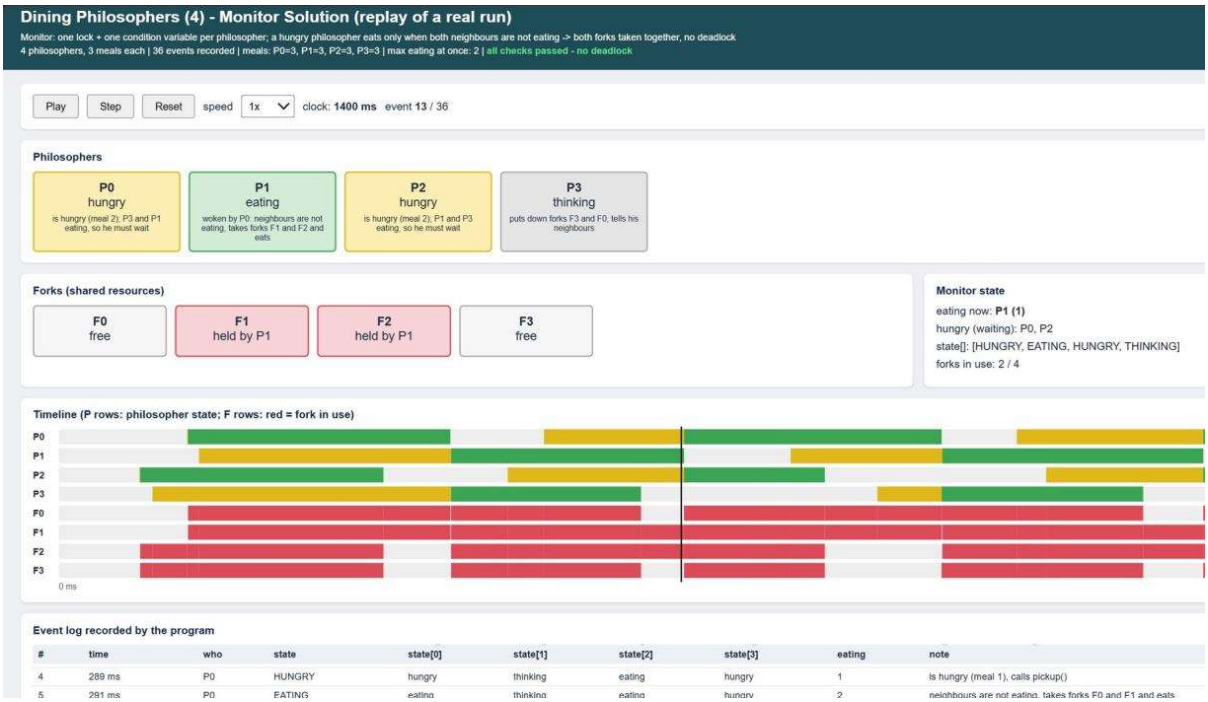


Figure 4: *dining_philosophers_monitor.html (event 13 of 36, both forks taken together)*

11. AI Use Summary

Item	Description
AI tool(s) and model(s)	Claude (Anthropic) [5] was a resource which assisted in comprehending the structure of the program, concepts of synchronization, and writing this report. I tested and executed these programs myself.
Initial prompt for the solution	The following four programs were asked for: Readers-Writers program using Semaphores; Readers-Writers program using Monitor (`threading.Condition`, Writer Priority);
Important follow-up prompts	and the Dining Philosophers problem for four philosophers using both approaches. Each program is required to record all the actions taken and produce an HTML animation based on the information collected.
What I changed manually	I chose the scenarios myself (four readers and two writers, five readers in monitor scenario, four philosophers and three meals each), ensured that all problems were in their separate pair of files, ran each program several times, and analyzed the code until I could explain each acquire, release and notify call. In addition, I made a conscious choice to explicitly state that Python semaphores and conditions are not FIFO, so starvation is possible.
My own testing	Testing has been carried out on all four implementations, the HTML events have been analyzed and compared with the terminal output (counts, end values, fork ownership). All automatic tests passed successfully: 8 out of 8 for the Readers-Writers monitor and 7 out of 7 for each case of Dining Philosophers problem.
Checking the AI output	However, they were not accepted blindly without any verification: They were crosschecked with the textbook as well as with the articles from the references [2][7][8] and also with the results obtained from running the recorded experiment (for example, four readers reading together and a writer coming in only when the last reader left).
Limitation found	Since the Python semaphore and condition variable are not first-in, first-out, then the thread could end up waiting forever even without deadlock avoidance.

12. References

[1] Python Software Foundation, “threading - Thread-based parallelism,” *Python 3 Documentation*. https://docs.python.org/3/library/threading.html

[2] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed., Wiley, 2018.

[3] A. S. Tanenbaum and H. Bos, *Modern Operating Systems*, 4th ed., Pearson, 2014.

[4] A. B. Downey, *The Little Book of Semaphores*, Green Tea Press. https://greenteapress.com/wp/semaphores/

[5] Anthropic, “Claude,” AI assistant. https://claude.ai

[6] E. W. Dijkstra, “Cooperating sequential processes,” EWD123, 1965.

[7] P. J. Courtois, F. Heymans, and D. L. Parnas, “Concurrent control with readers and writers,” *Communications of the ACM*, vol. 14, no. 10, pp. 667-668, 1971.

[8] C. A. R. Hoare, “Monitors: An operating system structuring concept,” *Communications of the ACM*, vol. 17, no. 10, pp. 549-557, 1974.

[9] E. G. Coffman, M. J. Elphick, and A. Shoshani, “System deadlocks,” *ACM Computing Surveys*, vol. 3, no. 2, pp. 67-78, 1971.

[10] W. Stallings, *Operating Systems: Internals and Design Principles*, 9th ed., Pearson, 2017.

13. Statement of Academic Integrity

> *I confirm that I have individually checked and comprehended the presented programs. I have revealed the use of AI programs and other sources for creating this assignment. I can provide the synchronization mechanism, generated HTML simulation, and changes in the source code. I have not deliberately copied someone else's work and submitted it as mine.*

Name: Geona Michelle	**USN:** NNM24IS085
Signature:	**Date:**

turnitin Page 22 of 22 - Integrity Submission Submission ID trn:oid:::1:3688300690